Fitra

Test Plan Report (IEEE 829 Format)

Version 1.0

Revision History

Revision Number	Date	Primary Author(s)	Comments
0.1	Oct 29th th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Initial draft
0.2	Oct 31th, 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Updating of draft
1.0	Nov 4th , 2026	Gary Quah, Lau Jing Jie, Lee Zheng Wee, Rusyaidi, Quah Yew Yi, Matthew Wong	Completion of edits for initial report.

Table of Contents

Revision History **2**
1. Test Plan Identifier **4**
2. Introduction **4**
3. Test Items (Functions) **4**
4. Software Risk Issues **4**
5. Features to be Tested **5**
6. Features not to be Tested **5**
7. Approach (Strategy) **6**
7.1 Test baseline and traceability 6
7.2 Test levels and tools 7
7.3 Test design and feature coverage 7
7.4 Environment and integration controls 8
7.5 Performance and execution records 9
7.6 Regression and defect management 9
8. Item Pass/Fail Criteria **9**
8.1 Individual test results 9
8.2 Functional acceptance criteria 10
8.3 Coverage and performance gates 11
8.4 Defect severity and completion 11
9. Suspension Criteria and Resumption Requirements **12**
9.1 Suspension criteria 12
9.2 Acceptable defect levels 13
9.3 Fatal errors and ghost errors 13
9.4 Resumption requirements 13
9.5 Decisions and records 14
10. Test Deliverables **14**

11. Remaining Test Tasks..... 16
12. Environmental Needs..... 17
13. Staffing and Training needs..... 17
14. Responsibilities..... 18
15. Schedule..... 20
16. Planning Risks and Contingencies..... 20
17. Approvals..... 21

1. Test Plan Identifier

The identifier for this system test plan for the Fitra web application project is Fitra-TPLV1.0-SL1-MTP1. The plan contains information about the resources, procedures and plans for testing Fitra.

2. Introduction

This is the Master Test Plan for the Fitra web application project. The primary focus is to ensure the application runs smoothly and fulfils all functional requirements mentioned in the Software Requirements Specifications without any bugs.

Testing will be done through the following:

a. **Component level** to ensure all components are working as intended, and
b. **End-to-end** manual testing to ensure that user interactions are working as intended
c. **User acceptance** testing from a closed group of users who have not interacted with Fitra before to observe for any unexpected behaviours or improvements to the application from the target user demographic.

3. Test Items (Functions)

A. Fitra Software, initial version to be Version 1.0
B. Fitra Systems Requirement Specification document
C. Fitra Quality Plan document
D. Fitra Project Plan document
E. Fitra Software Configuration Management Plan document

4. Software Risk Issues

The following software risk issues have been identified for Fitra:

1. **User Authentication and Access Control:** Incorrect authentication or access control may allow unauthorised users to access protected pages or personal health information.
2. **Database Operations and Data Integrity:** Errors when retrieving, inserting, updating or deleting records may result in inaccurate, missing or duplicated user data.
3. **Friends Management:** Errors in sending, accepting, rejecting or removing friend requests may result in incorrect friendship statuses. Duplicate or simultaneous requests may also cause inconsistencies.
4. **Data Privacy and Security:** Inadequate access control may expose personal information to other users. Appropriate access restrictions must be verified to ensure that users can only access data they are authorised to view.

5. **Integration Between Components:** Interactions between the frontend, authentication services and database may fail, resulting in unexpected behaviour or unsuccessful operations.
6. **Requirement Misinterpretation:** Unclear or misunderstood requirements may cause the application to behave differently from what users expect. Test cases should therefore be reviewed against the documented requirements.
7. **Error Handling and Input Validation:** Insufficient validation or error handling may allow invalid inputs or cause the application to fail when operations cannot be completed.

These risks will be addressed through functional testing, input validation, access control verification and testing of normal and exceptional scenarios. Higher-risk areas will receive greater testing attention.

5. Features to be Tested

The following is a list of Fitra features to be focused on during the testing of the application.

Features	**Level of Risk (H, M, L)**
Register Page	M
Login Page	M
Friends Page	L
Home Page	M
Settings Page	L
Training Page	H
Nutrition Page	H
Leaderboard Page	M
AI Assistance Page	H

6. Features not to be Tested

The following exclusions apply to the current FITRA release. They limit what this test cycle can demonstrate; they do not remove the need to test FITRA's own integrations, validation, access controls or recovery behaviour.

Excluded area	Reason and testing boundary
Internal operation of external platforms	Supabase, Gemini, Strava and the hosting providers are managed externally. Their internal algorithms, infrastructure and uptime are excluded. FITRA's authentication, token refresh, data mapping, timeout handling and responses to unavailable services remain in scope.
Live wearable streaming and additional import formats	Live synchronisation from wearable hardware and health export formats outside the agreed SRS are excluded from the release scope. Test supported imports and Strava account linking using the approved interface requirements.
Clinical outcomes and exact AI estimates	Clinical diagnosis, injury prevention guarantees and laboratory accuracy of photo-based nutrition estimates are outside this fitness application's purpose. Test configured training safeguards, numerical calculations, structured AI output, review and correction of estimates, and manual logging.
Future enhancements	Features proposed during user acceptance testing but deferred to a later release are excluded from this cycle. Record them through change control; include approved current-release requirements in the test matrix.
Unsupported client configurations	Native mobile applications, offline operation and obsolete browsers are excluded from this web release. Desktop and mobile browser layouts remain in scope under the configurations in section 7.
Large-scale and destructive exercises	Exhaustive device combinations, sustained high-volume stress or endurance runs, and destructive recovery tests on shared or production services are excluded from this cycle. Measure normal-use performance and test isolated failure scenarios. Record capacity and disaster-recovery risks for release review.

QA shall record each exclusion against the affected requirement and residual risk. The Project Manager and QA Manager shall review exclusions before test execution and approve any later scope reduction through the change-control process. An unavailable service is a blocked integration test, rather than a new exclusion or a pass.

7. Approach (Strategy)

FITRA will use risk-based testing across components, APIs, integrated user journeys and user acceptance. Authentication, personal-data isolation, training safeguards and correctness of saved health, nutrition and XP records receive priority. Testing covers normal use, invalid input, boundary values, repeated actions and service failures.

7.1 Test baseline and traceability

Before execution, identify the release commit, database migration set and approved SRS version. Complete and review the feature list and risk ratings in section 5. Each test case shall record its requirement reference, priority, preconditions, data, steps and expected result. Use the requirement title and document tab alongside the identifier where IDs are duplicated. Resolve conflicting requirements with the Project Manager and Lead Developer before treating either behaviour as the acceptance baseline.

Tests shall verify the agreed data boundary for Strava: imported activities may be displayed or saved to the schedule, but must not influence readiness, training calculations or AI coaching context. Verify this using payload inspection and the existing Strava-isolation tests. Any requirement describing a different data path shall be reconciled through change control.

7.2 Test levels and tools

Level	Execution and evidence
Component and unit tests	Use Vitest with React Testing Library and jsdom for frontend behaviour and request handling; use pytest for backend rules, calculations and services. Reuse the existing suites and deterministic fixtures.
API and service integration	Use FastAPI TestClient/httpx and existing dependency overrides for valid requests, authentication failures, ownership checks, validation and provider errors. Run separate checks against an isolated Supabase test project to verify real SQL functions, constraints, transactions and row-level security.
Manual system tests	Exercise complete browser journeys with the frontend connected to the real backend and test database. Cover registration and login, profile updates, friends, daily check-in, training, meal logging, AI assistance and gamification. Verify saved records and dashboard totals after refresh.
User acceptance tests	Ask a closed group of users unfamiliar with FITRA to complete representative journeys using prepared test accounts. Record task completion, observed problems and feedback. Defects in agreed requirements require correction and retesting; enhancement requests follow change control.

From frontend/, run npm test, npm run typecheck and npm run test:coverage. From backend/, run python -m pytest tests/ --cov=app --cov-branch --cov-report=term-missing --cov-report=html:htmlcov when pytest-cov is available in the test environment. Retain logs and coverage reports. Run configured lint/static checks, and the frontend production build before release; a missing or incompatible check is a recorded tooling issue, not a successful result.

Testers need a short walkthrough of these commands, report locations, mock-versus-HTTP configuration, test-account setup and defect recording. Live-service tests require separate credentials and test data; automated unit tests shall use mocked provider responses rather than depend on shared services.

7.3 Test design and feature coverage

Use equivalence partitions and boundary values for profile measurements, training preferences, check-in ratings, dates and nutrition fields. Include empty, malformed, negative, non-finite and excessive values. Use fixed dates for streaks, calendar totals and readiness so results are repeatable. Prepare two users, an unrelated user, fresh and returning profiles, empty and populated histories, and expired or missing sessions.

Training cases shall cover plan generation and replacement, readiness classification, accept/swap/skip, workout logging, rescheduling and recalculation. Check the configured weekly load limit and prohibition on consecutive high-intensity sessions. Exercise readiness boundaries at 44/45 and 69/70, insufficient history, missing check-in or sleep data, and trained-model fallback.

Nutrition cases shall verify acceptance of valid JPEG, PNG, WEBP, HEIC and HEIF images up to 10 MB and server-side conversion to JPEG before analysis. Include HEIC/HEIF files with missing browser MIME types, using their file extensions to set the upload MIME type.

If the browser cannot display a preview, verify that the filename appears and analysis remains available. Verify rejection of PDF, GIF, BMP, corrupt images and uploads over 10 MB. Also cover optional notes, no-food or malformed AI responses, ingredient edits, portion scaling, manual entry, confirmation and deletion. Confirm that analysis alone saves nothing, totals reflect reviewed ingredients, and retrying the same save does not create duplicate meals. Verify that portion scaling and draft totals update without network requests, and that confirmation validates reviewed ingredients and derives saved totals server-side rather than accepting a client-supplied aggregate.

Cross-feature cases shall verify health calculations and dashboard summaries; friend request states and restricted challenge invitations; and one shared daily questionnaire across Home and Gamification.

Verify that Home opens the questionnaire without awarding XP; all three 1–5 ratings are required; and only a successful submission records the check-in. Exercise Singapore midnight boundaries, a first check-in, consecutive days and a missed day. Verify 10 XP on day 1, 12 XP on day 2 and a 22 XP cap from day 7, with no extra XP or streak increment for a same-day repeat, retry or concurrent submission. Verify that sleep entry on Training does not complete a check-in. Also cover XP awarded once per eligible source, pending-XP resynchronisation and leaderboard visibility limited to the user and accepted friends. Test UI loading, empty and error states alongside successful operations.

7.4 Environment and integration controls

Use one isolated test deployment with the release migration set and three recorded client configurations: Chrome on Windows at a desktop viewport, Edge on Windows at a desktop viewport, and Chrome mobile emulation at a narrow viewport with touch input. Record browser versions, viewport, backend/frontend versions and network conditions. Mobile emulation does not establish compatibility with every physical device. Verify navigation, forms, tables, calendars, photo selection and error messages in each configuration.

Run frontend training and gamification clients in HTTP mode for integrated tests; retain mock mode for isolated component tests. Use dedicated Supabase accounts, Gemini credentials and an authorised Strava test account. Verify OAuth consent/cancellation, token expiry, repeated imports and user separation. Simulate rate limits, timeouts and malformed responses, then perform controlled live-service smoke tests. A mock result cannot replace the corresponding live integration result.

Check that browser-accessible requests cannot read or alter another user's records and cannot expose service-role keys or tokens. Backend ownership checks remain necessary because service-role database access bypasses row-level security. Test database RLS separately with ordinary authenticated clients.

7.5 Performance and execution records

Measure internal endpoint latency, live AI response time, initial meaningful page load and client-side nutrition recalculation against the Quality Plan and SRS targets in section 8. For each path, use ten recorded normal-use runs under stated network conditions, separating cold-start and warm runs and recording payload sizes. Report individual results, median and maximum. Record provider delays separately while retaining the full user-visible latency; mocked timings do not demonstrate live performance.

At each test level, collect planned, executed, passed, failed, blocked and not-run counts; requirement coverage; defects by severity; and retest outcomes. Collect line and branch coverage separately for frontend and backend. Report pass rate as passed cases divided by all planned in-scope cases, so blocked or unexecuted cases cannot inflate it.

7.6 Regression and defect management

After a change, rerun the affected automated suites and related API tests. Changes to authentication, shared models, database migrations or cross-feature calculations require the full automated suites and relevant end-to-end journeys. Before release, run all automated suites and repeat the core browser smoke tests on the release commit.

Record defects in the team's JIRA project with requirement, environment, reproducible steps, expected/actual result, severity and evidence.

Review progress and blockers during team check-ins; escalate high-severity issues immediately to the Project Manager and QA Manager. QA confirms fixes with the original case and related regression cases. Version test cases and results with the release; redact personal data and credentials from retained evidence.

8. Item Pass/Fail Criteria

These criteria define the expected outcome of test execution. They are acceptance requirements for review and do not indicate that testing has already passed.

8.1 Individual test results

Pass: all expected outcomes are observed on the recorded build, including UI feedback, response status/schema, persisted records, calculations and access restrictions where applicable. No unexpected duplicate, data loss or cross-user disclosure occurs. Attach sufficient evidence to reproduce the result.

Fail: an observed result differs from the approved requirement or expected outcome. Record a defect and the evidence. A partial success, an unhandled error or a successful page render with incorrect saved data is a failure.

Blocked: execution cannot establish an outcome because a required service, account, migration, dependency or approved requirement is unavailable. Record the cause and retest after resolution. Not run: execution has not started. Neither status counts as a pass. Apply section 9 when a blocker invalidates further testing.

8.2 Functional acceptance criteria

Test item	Required outcome
Accounts and access	Valid registration, login and logout complete as specified. Invalid credentials and expired sessions are handled. Protected pages and APIs reject unauthorised access, and user A cannot read or modify user B's private data.
Profile and dashboard	Valid profile changes persist and survive refresh. BMR, TDEE, BMI and relevant summaries match independently calculated fixtures using the approved formulas and rounding. Invalid values are rejected without changing saved data.
Friends and challenges	Requests, acceptance, rejection and removal produce consistent states. Unrelated users cannot join private challenges or appear through unauthorised access. Progress and rewards follow the defined dates and participation rules.
Training and readiness	Plans match preferences and configured safety rules. Readiness classes respect the configured thresholds: deload below 45, light from 45 to 69, and push from 70. Deload days do not recommend high intensity; logging, rescheduling and recalculation retain consistent session history.
Meal analysis and nutrition	Approved uploads produce a valid editable estimate or a clear recoverable error. Nothing is saved before confirmation. Reviewed ingredients, portion calculations, daily totals and deletion are correct. Invalid saves leave no partial records; identical save retries create one meal.
Check-in and gamification	One questionnaire submission per local day updates the shared calendar, streak and XP once. Duplicate submission is rejected. Workout/meal awards and challenge rewards are idempotent; failed awards remain recoverable as pending XP. Strava-sourced logs earn no workout XP.
Strava integration	OAuth, token refresh, permitted retrieval, repeated imports and cancellation behave as specified. Each user sees only authorised activity data. Imported data stays outside readiness, training calculation and AI payloads; repeated saves do not duplicate schedule entries.
AI assistance and resilience	Valid requests return usable, correctly attributed output. Blank/invalid input is rejected; timeouts, rate limits and provider failures show a useful message. Manual logging and unrelated application functions remain usable. AI output cannot silently overwrite confirmed user records.

8.3 Coverage and performance gates

The Quality Plan requires at least 80% line coverage and 75% branch coverage for both frontend and backend, and a 100% pass rate on production releases. Measure these against the agreed application source scope and retain the reports. Do not exclude untested application modules merely to meet the targets. Code coverage supplements requirement coverage; it does not replace manual, database or live-service testing.

Measurement	**Acceptance target and method**
Internal API latency	Under 300 ms for internal endpoints such as profile and daily-log retrieval, using the normal-use measurements in section 7.5.
Live AI latency	Under 3 seconds for meal analysis and chat responses, measured through the real integration. A graceful timeout does not satisfy this latency target.
Initial page load	Under 1.8 s from navigation to the first meaningful DOM content, measured in the recorded browser/network configuration.
Nutrition recalculation	Under 50 ms for client-side portion and macro recalculation, without a server round trip, as specified in the SRS.
Meal-analysis timeout	A stalled AI call is terminated at the SRS server-side limit of 10 s and produces the defined recoverable error. Verify separately from successful-response timing.

Use the maximum of the ten recorded runs for each tested path as the review gate, and show all measurements so variability is visible. A missed target remains an open non-functional issue until corrected or formally changed through the Quality Plan/change-control process. A slow external provider shall be documented rather than removed from the user-visible measurement.

8.4 Defect severity and completion

Critical: unauthorised disclosure or modification, lost/corrupted records, or a training recommendation that violates configured safety limits. High: a core journey cannot be completed, or important calculations/rewards are wrong without a viable workaround. Medium: limited incorrect behaviour with a usable workaround. Low: cosmetic or minor usability defects that do not change outcomes. QA assigns severity and confirms it with the feature owner.

The plan is complete only when all planned in-scope cases have been executed and passed on the release build, every approved requirement has linked evidence, the coverage and performance gates are met, and no Critical or High defects remain open. All fixes require successful retesting and relevant regression testing. UAT feedback must be reviewed and required corrections verified.

Any proposed acceptance of an unresolved Medium or Low defect, missed metric or deferred case requires a documented impact assessment and approval by the Project Manager and QA Manager through change control. Update the baseline scope or criterion before issuing the final report; a failed or blocked case cannot simply be relabelled as passed. Retain the original result and decision.

The final test summary shall identify the release commit, environment, test totals, requirement and code coverage, performance measurements, unresolved risks and approved deviations. Submit it for the approvals in section 17. Source signatures or dates retained in this review copy do not approve the revised sections.

9. Suspension Criteria and Resumption Requirements

Testing is suspended when the number or type of defects leaves further testing with no value, or when its results would not describe the system under test. Going on would waste resources on failures caused by an earlier defect. Defects are rated with the Change Management Plan's priorities: Critical, High, Medium and Low.

9.1 Suspension criteria

A single test stops when a step fails and its later steps depend on that step. The remaining steps are marked blocked, the defect is logged, and the next test case runs. The criteria below stop a series of tests: a feature's cases, or the whole test cycle.

#	Suspend testing when	What stops	What can continue
S1	The commit under test fails the production build or the TypeScript check, or a test suite cannot be collected	All testing on that commit	Nothing
S2	Anything other than an approved fix is merged into main during a test cycle	The cycle, because its results no longer describe one version	Nothing
S3	The backend cannot be reached, or `GET /health` reports that the database is not connected	Cases for Home, Settings, the chatbot, Training, Meal analysis and Gamification, which all call the backend	Login, registration and Friends, which use Supabase directly; automated unit tests
S4	A migration is being applied to the shared Supabase project	Every test that writes data: check-ins, workouts, meals, friend requests and chat messages	Unit tests that do not use the database
S5	The test environment is set up wrongly: `NEXT_PUBLIC_TRAINING_API` or `NEXT_PUBLIC_GAMIFICATION_API` is not `HTTP`, or a key is missing on Vercel or Render	Cases for the affected features (Training and Leaderboard would show built-in sample data)	Other features
S6	Gemini or Strava is down or over its free-tier quota	Meal analysis, coach and chatbot cases (Gemini); Strava tab and account-link cases (Strava)	Other features. Manual runs against sample data are logged but do not count as a pass
S7	A Critical defect is found: login fails, one user can see another user's data, Strava data reaches an AI feature, or a secret key is exposed	All testing. A login failure blocks every dashboard page, and a data or key exposure must be contained first	Nothing

S8	25% or more of one feature's test cases fail in a run, or three High defects are open on it	That feature's remaining cases, and those of the features built on it. Home shows every feature, Gamification uses Training, Meal analysis and Friends, and the chatbot adds sessions to Training	Other features

9.2 Acceptable defect levels

Testing can proceed past a defect when three things hold: it is logged, the remaining cases do not depend on the failed step, and open defects stay within the levels below. A defect logged before a run does not suspend testing by itself while these levels hold.

Priority	**Acceptable while testing continues**	**When the level is exceeded**
Critical	None	All testing is suspended (S7)
High	Up to two open on one feature, and none that blocks the feature's remaining cases	The feature's testing is suspended at the third (S8)
Medium or Low	Any number, while fewer than 25% of the feature's cases fail in the run	The feature's testing is suspended at 25% (S8)

9.3 Fatal errors and ghost errors

A fatal error is a failure that makes later results unreliable, such as a failed build (S1), a lost backend or database (S3) or a Critical defect (S7). Testing stops as soon as one is found, because later failures may be ghost errors: they look like defects but are caused by the first one.

Results recorded after a fatal error do not count. Those failures go in the test log as suspected ghost errors, not as new defects. The cases are rerun after the fix, and only failures that remain are reported as defects.

9.4 Resumption requirements

Every resumption needs three things:

- The cause is fixed and recorded: a closed Jira defect, or a note in the test log for an outside outage.
- `GET /health` reports a connected database, and a test account can log in.
- The cases that failed or were cut short are rerun from their first step.

#	**Resume when**	**Retest on resuming**
S1	A new commit passes the build, the type check and test collection	The whole cycle, on the new commit
S2	The new commit is recorded and main is frozen again	Both automated suites in full, plus manual cases for the changed features and the features built on them.

		Other manual results carry over only if their code did not change
S3	`GET /health` reports healthy. A slow first response after Render has been idle is not a failure	The suspended cases, from the start
S4	The migration has finished, its owner confirms it, and the features using the changed tables pass a smoke test	The write tests, including any that were cut short
S5	The setting is corrected, the app is redeployed, and the pages show live data	The affected cases
S6	The service responds again or its quota resets	The affected cases, on the real service
S7	The fix is merged through an approved change request and the defect is closed after a retest. For an exposed key, the key is also rotated on Vercel and Render and the app redeployed	Full regression: both automated suites, and every manual case of the affected feature and the features built on it
S8	The fixes are merged and the failed cases pass on retest	The feature's full set of cases, and those of the features built on it

9.5 Decisions and records

The QA Manager (Lau Jing Jie) suspends and resumes testing. During integration and system testing, its lead (Quah Yew Yi, Project Plan task X07) may also suspend a run, and anyone may stop testing for a Critical defect (S7).

Each suspension and resumption goes in the test log with:

- the date and time
- the commit
- the trigger
- the cases affected
- who decided
- the evidence for resuming

The Release Manager is told at once, because a suspension close to a release can delay it. If a suspension is not cleared by the release cut-off, two days before the release, the QA Manager takes it to the CCB. The CCB then either moves the release, which needs all three members to agree, or releases with the affected feature switched off.

10. Test Deliverables

The test deliverables are the documents, code and records that testing produces. The Configuration Management Plan places four of them in the Lab 5 product baseline: the test plan, the test cases, the coverage report and the requirements test. Fitra itself is not a test deliverable: it is a test item (section 3), delivered by development.

Deliverable	Content	Produced by	Format and location	When

Test plan document	This document: scope, approach, criteria, schedule and responsibilities	QA Manager, with the team	Google Docs, with a PDF in the project documentation folder	Before testing starts; revised when the scope changes
Test cases	For each case: ID, name, description, pre-conditions, post-conditions and assignee, then each step with its expected and actual outcome	QA Engineers, with feature owners	Test Coverage Report, section 2.1	Before each test cycle; actual outcomes added after each run
Test design specifications	For each feature: what will be tested, the techniques used (equivalence partitioning, boundary values, state transitions) and the test cases that carry them out	QA Engineers, with feature owners	Test Coverage Report, before the test cases	Before the test cases are written
Tools and their outputs: automated test suites	Backend Pytest suites (training, gamification, the Leaderboard page, nutrition, Strava, dashboard) and frontend Vitest suites (dashboard, nutrition), with their pass or fail results	Feature owners	Code in backend/tests/ and frontend/tests/ on main	With each pull request
Tools and their outputs: coverage reports	Line and branch coverage from pytest-cov (backend) and Vitest with V8 coverage (frontend), against the Project Plan targets of 80% of lines and 75% of branches	QA Engineers	HTML in backend/htmlcov/ and frontend/coverage/. These folders are git-ignored, so the reports are exported and attached to the test summary report	End of each test cycle
Simulators	Stand-ins for parts of the system. training_fakes.py and the in-memory Supabase in test_gamification.py replace the database, and the web app's mock training and gamification APIs replace the backend	Feature owners	Code in the repository	With the features they support
Static and dynamic generators	Static: fixed test data in the tests and mock APIs, such as test accounts, check-in answers and dates. Dynamic: data built during a run, such as workout history from seed_history in training_fakes.py, generated IDs and the frozen server clock. No third-party data generator is used	Feature owners	Code in the repository	With the tests that use them

Error logs and execution logs	Execution logs: for each run, the date, commit, tester, environment, and pass or fail for each case. Error logs: stack traces from failed test runs, plus Render service logs and Vercel build logs for the run	Whoever runs the cycle	Saved Pytest and Vitest output, the manual run sheet, and log extracts attached to the run	Every run; error logs when a run fails
Problem reports and corrective actions	Problem report: ID, severity, steps to reproduce, commit, expected and actual result, and status. Corrective action: the fix, its pull request and the retest that closed the report	Testers; corrective actions by feature owners	Jira issues	When found; closed after a passing retest
Requirements test	Each of the 30 SRS use cases mapped to its test cases and latest result. H-1 implemented; automated tests passed	QA Engineers	Table in the Test Coverage Report	End of each test cycle
Suspension and resumption log	Each suspension in section 9: trigger, cases affected, decision, evidence for resuming, and the rerun result of any suspected ghost errors	QA Manager	Part of the test log	When it happens
Smoke test record	The release-day checks: each feature used once on the live site, plus /health, the coach and meal analysis endpoints	Release Manager and feature owners	Checklist attached to the release's Version Delivery Description	Each release
Test summary report	What was run, pass rate, coverage, open defects by severity, suspensions, and a go or no-go recommendation for the release	QA Manager	Document, given to the Project Manager, who authorises delivery	End of each test cycle, before the release is tagged

11. Remaining Test Tasks

This test plan covers the core functionalities of the Fitra web application, with reference to the functional requirements defined in the Software Requirements Specifications (SRS) document, including user registration and login, user health data tracking, nutritional data tracking, AI assistance, leaderboard system and training data tracking.

Any additional features identified from user acceptance testing will not be included in the current release and will be excluded from this round of testing. When these features are implemented in

future development phases, additional test cases will be developed to verify the new features function as intended.

Re-testing of features prior to implementation of new features will also be required to ensure no unintended behaviours arise as a result of implementing these new features through regression testing.

12. Environmental Needs

For the testing of Fitra, the following environment specifications and requirements have to be met:

a. **Platform:** A Personal Computer (PC) such as Windows, Linux or macOS will be required to interact with the user interface at an appropriate resolution.
b. **Web browser:** A modern web browser supporting JavaScript, such as Chrome, is required to access and test the Fitra web application.
c. **Stable internet connection:** A stable internet connection is required to access the deployed application and communicate with the backend services.
d. **Test data:** Test data, including user account details, training records, nutritional records and health metrics, will be provided to verify the application's functionalities. Both valid and invalid inputs will be used to validate the handling of data.
e. **Backend and database:** The backend and database will be required to be accessible from the frontend of the application to ensure the application is able to communicate with respective services.
f. **AI services:** The AI service must be available when testing AI-related functionalities. If the service is unavailable, tests may need to be postponed, recorded as a failure or recorded as blocked.
g. **API services:** The required API services such as Strava must be available when testing features depending on Strava data importing. If the service is unavailable, tests may need to be postponed, recorded as a failure or recorded as blocked.
h. **Testing environment:** All required environment variables and service credentials must be configured in the test deployment environment.

13. Staffing and Training needs

Testing is staffed for existing project team and no additional personnel are engaged. The QA manager (Lau Jing Jie) owns the test process and this plan. The QA Engineers prepare and execute the test cases, record results and raise defects. Integration and system testing is led by Quah Yew Yi under Project Plan task X07. Feature owners support testing of their own pages, confirm defect severity with QA and supply corrections for retest. Where test execution becomes a bottleneck, developers are cross-allocated to run and verify test suites, as set out in Section 16.

The following training is required before execution begins:

a. **Application walkthrough.** All testers require a walkthrough of the eight Fitra pages and the agreed behaviour of each, taken from the SRS. This covers the readiness classification, the Mifflin-St Jeor calculations, the progressive overload threshold and the Strava data boundary described in Section 7.1, since these determine whether an observed result is actually correct rather than merely plausible.
b. **Test tooling.** Testers require a session on the tools and commands listed in Section 7.2: Vitest with React Testing Library and jsdom, pytest, FastAPI TestClient and httpx, and the isolated Supabase test project. The session covers `npm test`, `npm run typecheck` and `npm run test:coverage` from `frontend/`, the pytest coverage command from `backend/`, where logs and coverage reports are written, how to switch between mocked provider responses and live HTTP calls, and how to configure test accounts and seeded data.
c. **Defect recording.** Testers require training on the defect workflow: recording defects in Jira against the affected requirement or module, applying the severity definitions in Section 8.4, and the escalation route for Critical defects.

Training is provided within the team. The QA Manager coordinates it and delivers the defect workflow and severity training. The Lead Developer (Matthew Wong) delivers the tooling session and the test harness configuration. Feature owners brief testers on their own pages. No external or vendor training is required and no training budget is allocated.

The software risk issues in Section 4 and the feature risk ratings in Section 5 determine the depth of training required for each area, and the schedule in Section 15 determines when it is delivered. Training take place before the first execution cycle of each test level and it repeated only where the tooling or the agreed behaviour changes.

14. Responsibilities

The QA manager is accountable for this plan as a whole. Specific responsibilities are allocated as follows.

Responsibility	Owner	Reference
Setting and maintaining test risks	QA Manager, with the Project Manager	Sections 4, 16
Selecting the features to be tested and not tested, and their risk ratings	QA Manager, with the Lead Developer and feature owners	Sections 5, 6
Setting the overall test strategy and test levels	QA Manager	Section 7
Approving the test baseline: release commit, migration set and SRS version	Lead Developer, confirmed by the QA Manager	Section 7.1

Ensuring environments, credentials and test data are in place	Release Engineer/Manager	Section 12
Preparing and executing test cases and recording results	QA Engineers	Sections 7.3, 10
Leading integration and system testing	Quah Yew Yi, Project Plan task X07	Section 9.5
Assigning defect severity	QA Engineers, confirmed with the feature owner	Section 8.4
Correcting defects and supplying fixes for retest	Feature owners, under the Lead Developer	Section 8.4
Suspending and resuming testing	QA Manager; the integration and system test lead within that level; any tester for a Critical defect	Sections 9.1, 9.5
Resolving scheduling conflicts and reprioritising test effort	Project Manager	Section 15
Providing training	QA Manager, Lead Developer and feature owners	Section 13
Reporting test status and open defects to the Project Manager	QA Manager	Quality Plan, Problem Reporting and Corrective Action
Accepting an unresolved Medium or Low defect or a missed metric	Project Manager and QA Manager, through change control	Section 8.4
Go/no-go decisions on items not covered by this plan	Change Control Board: Project Manager, Lead Developer, QA Manager	Section 9.5

Scheduling conflicts : Testing is not performed on a production system. The deployed environment is a release candidate rather than a live service, so conflicts arise between test execution and continuing development rather than between testing and users. Where a conflict arises, the Project Manager reassesses the timeline and reprioritises test effort towards critical functionality, as set out in Section 15.

Go/no-go decisions : The QA Manager makes the go/no-go recommendation for items covered by this plan, against the completion criteria in Section 8.4. For an item not covered by this plan, or where a suspension has not been cleared by the release cut-off two days before release, the decision passes to the Change Control Board. Moving a release requires the agreement of all three members; the alternative is to release with the affected feature switched off. The Release Engineer/Manager is informed of any such decision immediately.

15. Schedule

The testing schedule is established in the System Requirements Specification (SRS) to ensure that testing is a non-negotiable phase of the project plan. To prevent quality compromises, especially during crunch time, the schedule manages potential slippage through relative dating instead of strict deadlines. If any deliverables are delayed, the corresponding test will automatically adjust by the same duration. The team will implement, test, and refine features incrementally throughout the project lifecycle. Fitra's development activities are as stated in the project plan, however, despite the estimated duration and defined milestones, unexpected circumstances may also affect project's schedule. To minimise schedule slippage, Fitra will adopt the following measures.

a. **Reassessment of project timeline**
   The project manager will evaluate the extent of the delay and reevaluate development and testing activities accordingly.
b. **Prioritisation of essential features**
   Testing efforts will shift to prioritise critical system functionalities and features. Bugs will be classified according to severity levels and must be resolved before integrating into production. Minor bugs or issues may be moved to future improvements.
c. **Manpower Allocation**
   In the event of insufficient manpower or limited availability of QA personnel, developers may be tasked to assist QA team in executing test cases.

16. Planning Risks and Contingencies

The primary risks that can affect the test process are shortage of manpower for test execution, late delivery of software features, and limits imposed by third-party platforms such as Google Gemini and Strava. Contingency plans are in place to address these events. If requirements change or development slips, the Change Control Board (CCB) will be used to review the impact. Testing schedules will take place relative to the delivery date of the feature, or testing scope will be reprioritised using a risk-based testing approach focusing on core system components. To address testing bottleneck issues, the developer team are expected to support the QA team in test execution. The following risk log identifies several factors that can directly impact the test schedule and objectives.

Risk Event	Description	Mitigation Plan
Lack of testers	Accumulation of test backlog caused by staffing constraints resulting in verification bottlenecks and fatigue	Cross-allocate developers to run and verify test suites.

Late delivery of software features	Feature, branches, or build that extended past planned timeline, affecting testing window.	Apply relative dating to shift test start dates relative to the extended software feature delivery date.
Third-Party API Unavailability	External APIs (Gemini, Strava) service outage or free tier limits.	Switch environments immediately to a local mock JSON and seeded demo datasets.

17. Approvals

Review record: Gary Quah — scope, schedule and authority; Matthew Wong — technical controls and integration; Lau Jing Jie — quality gates and evidence. Resolve review comments before baselining.

Signature:	GQ	Date:	01/11/2026
Print Name:	Gary Quah		
Title:	Project Manager		
Role:	Plan approver		

Signature:	MW	Date:	01/11/2026
Print Name:	Matthew Wong		
Title:	Lead Developer		
Role:	Plan proposer		

Signature:	JJ	Date:	01/11/2026
Print Name:	Lau Jing Jie		
Title:	QA Manager		
Role:	Plan reviewer		